

Scientific Computing

L04 Complexity

Thorsten Koch

Applied Algorithmic Intelligence Methods department at ZIB
Chair of *Software and Algorithms for Discrete Optimization*
at TU Berlin



Topics 1: P, NP, undecidable

1. https://en.wikipedia.org/wiki/Computational_complexity_theory
2. https://en.wikipedia.org/wiki/Big_O_notation
3. What is Big O (Comparing Algorithms)
https://www.youtube.com/watch?v=MyeV2_tGqvw
4. P vs NP – An Introduction
<https://www.youtube.com/watch?v=OY41QYPI8cw>
5. P Formal Definition
https://www.youtube.com/watch?v=Of_cpDNoZtE
6. NP: How Non-determinism Relates to Verifiable Proofs
https://www.youtube.com/watch?v=FpeP_gr0Eu4
7. NP-Complete Explained (Cook-Levin Theorem)
https://www.youtube.com/watch?v=W9G_1xG77LE
8. Roughgarden: Algorithms Illuminated Chapter 2 (Part 1)
9. Roughgarden: Algorithms Illuminated Chapter 19, 23 (Part 4)

Topics 2: Tools

1. [TRPL Chap 5 Using Structs to Structure Related Data](#)
2. [The Rust Style guide](#)
3. [rustc - the Rust compiler](#)



8-bit CPU: Zilog Z80, at 1.76 MHz

Video: Monochrome

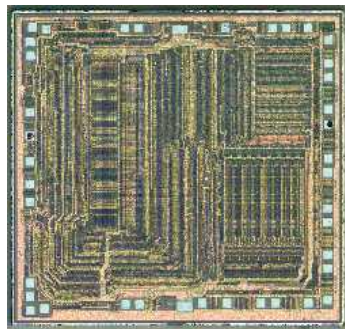
64×16 (uppercase) text, 128×48 block graphics

Composite video output, RF TV signal output

16 KB RAM, expandable to 48 KB

12 KB ROM containing Microsoft LEVEL II BASIC

Storage: Built-in 500 baud cassette deck



Z80 CPU: 8500 transistors



Speed-up of a modern computer regarding neural networks

Here is a comparison between my first computer and a system I currently use.

The calculation is a very rough estimate.

However, it is reasonable to say, that for particular applications, the modern system is about

~400 Billion times faster.

400 Billion times faster means:

80 years become 1 second

*dünnbesetzt

https://www.smxrtos.com/ussw/gofast/gofast_z80_z180_64180.pdf
<https://resources.nvidia.com/en-us-grace-cpu/grace-hopper-superchip>

	Z80	GH200	speedup
CPU Kerne	1	72	
L1 cache [KB]		64 + 64	
L2 cache [MB]		72 × 1	
L3 cache [MB]		114	
Taktfrequenz [GHz]	0,00176	3.1	1.761
RAM size [GB]	0,000048	480	10 Mil
FP64 [GFLOPS]	0,0000018	34,000	19 Bil
FP64 TC [GFLOPS]		67,000	
FP32 [GFLOPS]	0,0000061	67,000	11 Bil
FP32 TC [GFLOPS]	0.0000023	989,000*	430 Bil
FP16 TC [GFLOPS]		1,979,000*	
FP8 TC [GFLOPS]		3,958,000*	

Each of these tapes can hold about 2TB.
It continues around the corner at the end.

What do you think is
the fastest way to get
this data somewhere
else?

IBM TS4500 Taperobot with
6000 Slots => 30 PetaByte
in fireproof Lampertz IT-
Saferoom

Fully loaded with newest tapes
=> 300 PetaByte

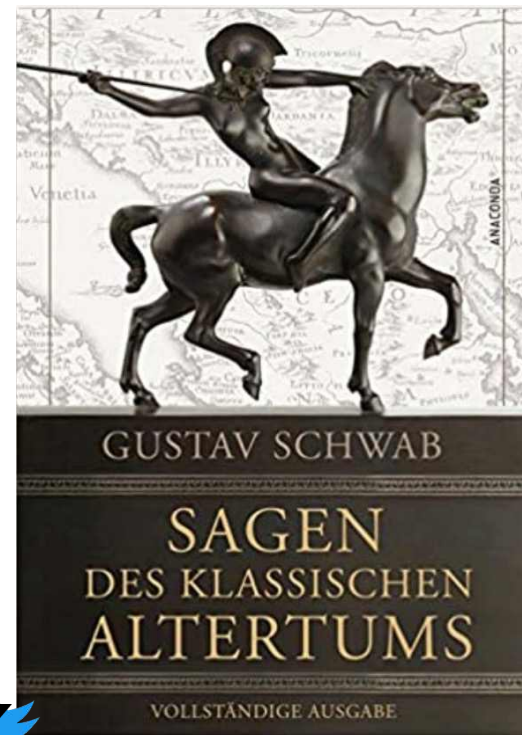
Extendable to 1.5 ExaByte



You can answer important questions like:

What is the percentage of sentences of
“Sagen des klassischen Altertums”
that fit into 140 / 280 characters?

Answer: 70% / 97%




```
// assert!(x >= 3 && x < 8); Use instead:  
assert!((3..8).contains(&x));
```

Any comparison with **NaN** is always **false**

```
assert_eq!(5/0, f64::NaN);  
assertion `left == right` failed  
  left: NaN  
 right: NaN
```

```
assert_eq!(expreval(&"12345.6789".to_string()), 12345.6789);  
assertion `left == right` failed  
  left: 12345.678899999999  
 right: 12345.6789
```

Never compare floating point numbers with equality. (unless you directly assigned the value you are comparing against.)

```
assert_eq!(expreval(&"0.123 * .001 * 2000000.".to_string()), 246.0);  
assertion `left == right` failed  
  left: 246.00000000000003  
 right: 246.0
```

Student presentation

Definition: Let $g: \mathbb{N} \rightarrow \mathbb{R}$.

f grows not much faster than g $f \in \mathcal{O}(g) = \{ f: \mathbb{N} \rightarrow \mathbb{R} \mid \exists \alpha > 0 \exists n_0 \in \mathbb{N} \forall n \geq n_0: 0 \leq f(n) \leq \alpha g(n) \}$

f grows not much slower than g $f \in \Omega(g) = \{ f: \mathbb{N} \rightarrow \mathbb{R} \mid \exists \beta > 0 \exists n_0 \in \mathbb{N} \forall n \geq n_0: f(n) \geq \beta g(n) \geq 0 \}$

f grows as fast as g $f \in \Theta(g) = \mathcal{O}(g) \cap \Omega(g)$

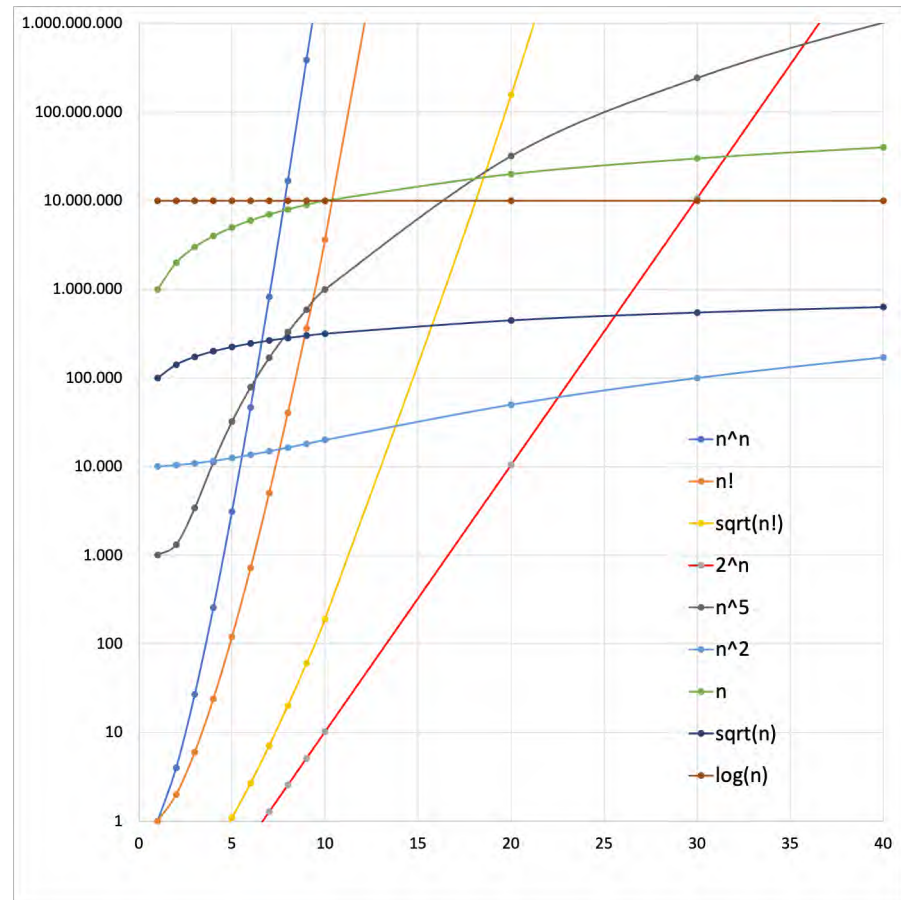
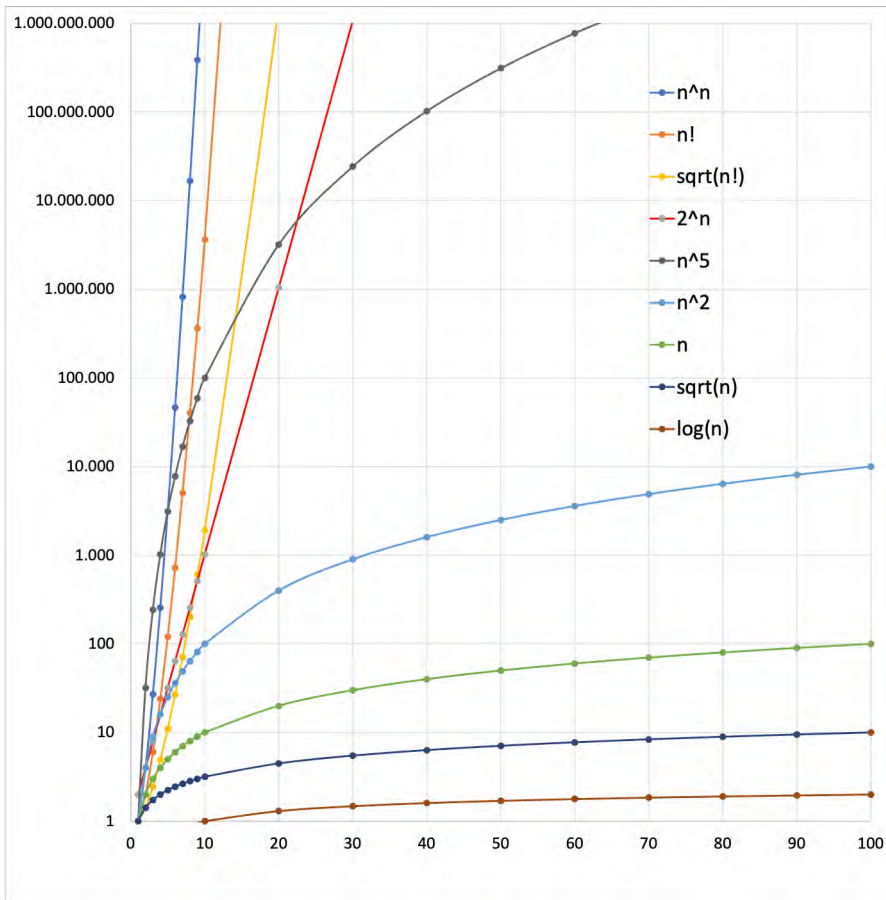
Speaking

- ▷ f grows at most linear $\Leftrightarrow f \in \mathcal{O}(n)$
- ▷ f grows at least quadratic $\Leftrightarrow f \in \Omega(n^2)$

Examples

- ▷ $3n^2 - 9n + 2 \in \Theta(n^2)$
- ▷ $|\sin(n \frac{\pi}{2})| \in \mathcal{O}(1)$ but $|\sin(n \frac{\pi}{2})| \notin \Omega(1)$
- ▷ $2^n \in \Omega(n^k)$ for any $k \in \mathbb{N}$

What $\mathcal{O}$ = Order of means: both pictures are equally correct



- ▷ The runtime of an algorithm for a specific input is the **number of elementary arithmetic operations** (e.g., comparisons, assignments, additions, multiplications, divisions, etc.) **that the algorithm performs for this input.**
- ▷ The runtime of an algorithm for any input is usually measured as a function of one or more input parameters; this is called the runtime function.
- ▷ Normally, we are only interested in the asymptotic behavior of the runtime function for large input parameters, which justifies the use of the $\mathcal{O}$ -notation (Landau symbols).
- ▷ What is the runtime complexity of the Collatz function? (Unknown)

Input: $n \in \mathbb{N}$

Output: Answer to the question whether n is prime.

1. if $n \leq 1$ then
 $e \leftarrow \text{false}$
else
 $e \leftarrow \text{true}$
2. for $i \leftarrow 2$ to $\lfloor \sqrt{n} \rfloor$ do:
 if i is a divisor of n then
 $e \leftarrow \text{false}$
3. output e

Observations

The algorithm returns the correct answer.

It has runtime $\mathcal{O}(\sqrt{n})$

- ▷ In 2002, Agrawal, Kayal and Saxena found a prime number test algorithm with running time $\mathcal{O}(\log n)^k$ for a constant k . Though AKS runs in polynomial time, it is too slow to be practical.
- ▷ Faster algorithms include the Miller–Rabin primality test, which is fast but has a small chance of error.
- ▷ No algorithm for factoring has been published that runs in polynomial time, i.e., that can factor a b -bit number n in time $\mathcal{O}(b^k)$ for some constant k . It is an open research question whether such an algorithm exists.
- ▷ For a quantum computer, Peter Shor discovered a polynomial time algorithm in 1994. It takes only $\mathcal{O}(b^3)$ time and $\mathcal{O}(b)$ space.

P “easy”	Solvable in polynomial time. A running time of $10^{20} \cdot n^{1000}$ for input size n is poly time.
NP	“Yes” answer can be checked in poly time. It does not matter where the answer comes from.
NP-Complete	Hardest problems in NP. If you could solve one of them in poly time, then you can solve them all.
NP-Hard “intractable”	As least as hard as the hardest problems in NP. Need not to be in NP.

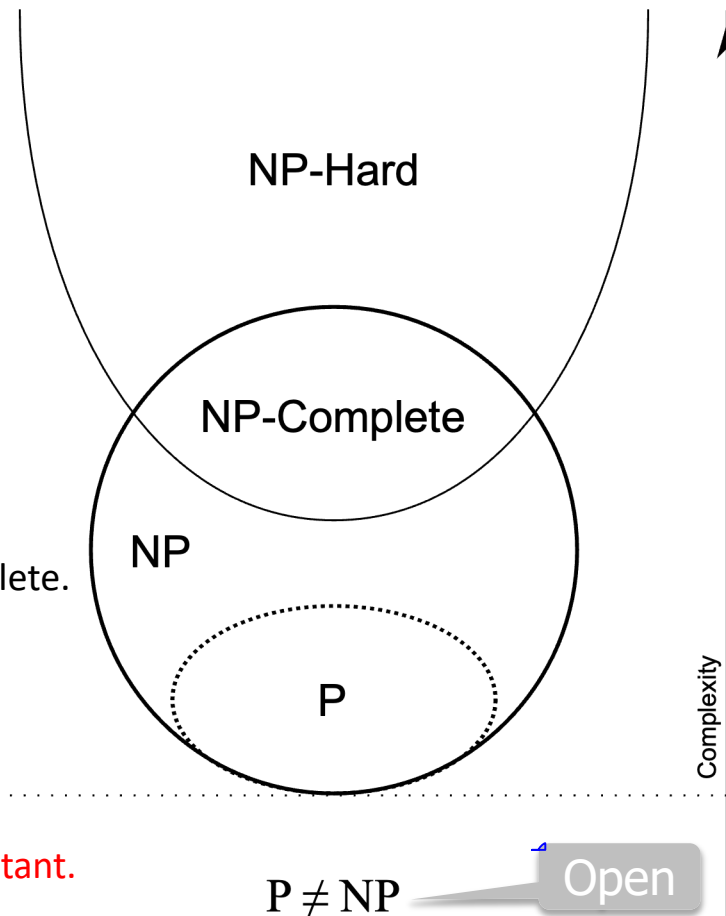
Integer factorization is in NP.

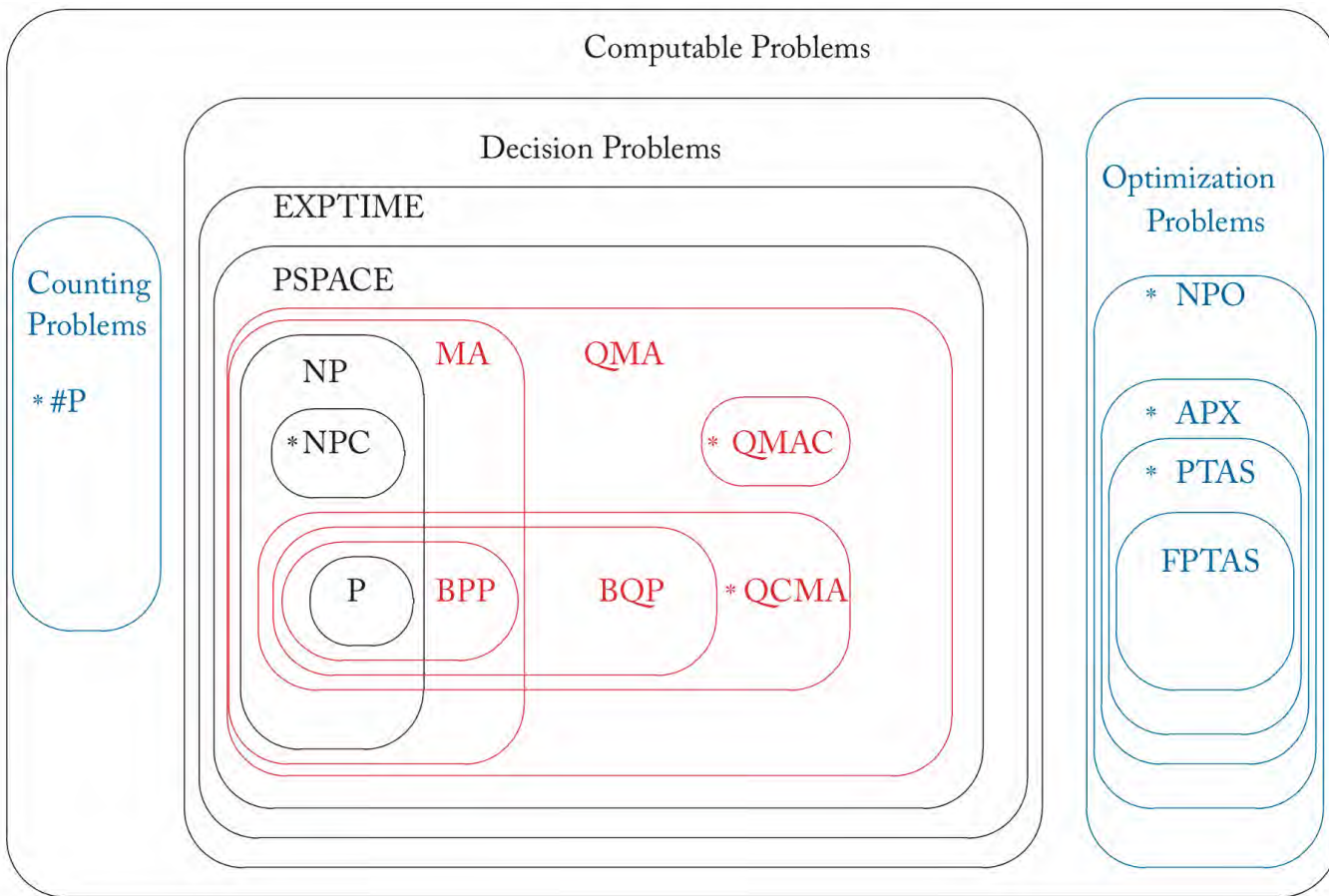
Current assumption: it is **not** in P and **not** NP-hard $\Rightarrow$ not NP-complete.

Integer Programming/Optimization is in NP.

Proven to be NP-hard and therefore NP-complete

A proof of $P=NP$ might not be constructive and include a huge constant.



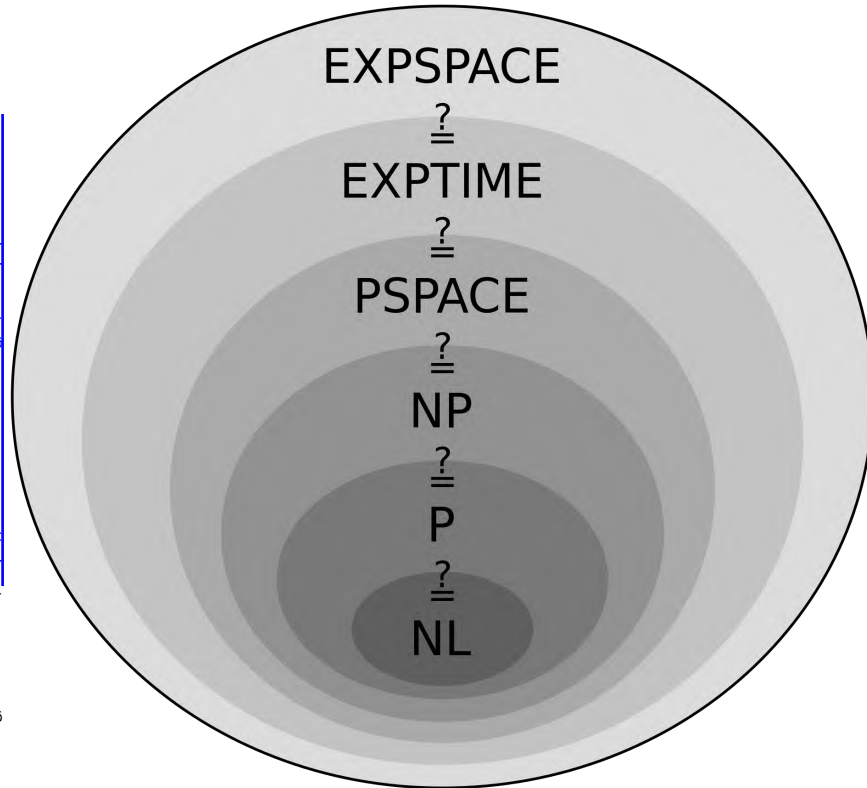


BQP is bounded-error quantum polynomial time. A quantum model of computation uses an arrangement of quantum circuits (in the gate model) operating on qubits. These problems can be solved by a quantum computer in polynomial time with an error probability of at most $1/3$.

QMA stands for Quantum Merlin-Arthur. This is the set of decision problems for which yes-instances can be verified by a quantum computer in polynomial time, and no-instances rejected in polynomial time, with error probability at most $1/3$. This is the quantum analog of NP, related to BQP as NP is related to P. QMAC is the set of complete problems for this class.

QCMA stands for Quantum-classical Merlin-Arthur. This class is similar to QMA but the witness for a yes-instance must be a classical string.

$$\text{NL} \subseteq \text{P} \subseteq \text{NP} \subseteq \text{PSPACE} \subseteq \text{EXPTIME} \subseteq \text{EXPSPACE}$$



- ▶ **NL (Nondeterministic Logarithmic Space):** Problems solvable by a nondeterministic Turing machine using only logarithmic space.
- ▶ **P (Polynomial Time):** Problems solvable by a deterministic Turing machine in polynomial time.
- ▶ **NP (Nondeterministic Polynomial Time):** Problems whose solutions can be *verified* by a deterministic Turing machine in polynomial time.
- ▶ **PSPACE (Polynomial Space):** Problems solvable by a deterministic Turing machine using polynomial space, regardless of time.
- ▶ **EXPTIME (Exponential Time):** Problems solvable by a deterministic Turing machine in exponential time (e.g., $2^{p(n)}$ for some polynomial p).
- ▶ **EXPSPACE (Exponential Space):** Problems solvable by a deterministic Turing machine using exponential space (e.g., $2^{p(n)}$ for some polynomial p).

The development of computational complexity theory has led, in the last fifteen years, to a fascinating insight into the inherent difficulty of combinatorial optimization problems, but has also produced an undesirable side effect which can be summarized by the “equation”

NP -hardness = intractability,

Summary:
This is wrong!

thereby diminishing attention to the study of exact algorithms for NP-hard problems. However, recent results on the solution of very large instances of integer linear programming problems with special structure on the one hand, and forty years of successful use of the simplex algorithm on the other, indicate the concrete possibility of solving problems exactly through worst-case exponential-time algorithms.

— Silvano Martello, Paolo Toth, 1989

Note: P = efficient – is also only generally true by the definition of “efficient” in the theoretical sense. Never forget: This is **WORST CASE** Complexity. The Average might be different. See, e.g., Quicksort.

Die Chancen der Quantenrevolution

Warum es sich strategisch lohnt, früh in die neue Schlüsseltechnologie zu investieren

S tellen wir uns vor, der Nikolaus wollte die kürzeste Route einer Rundreise ermitteln, auf der er Kinder mit Geschenken erfreuen möchte. Könnte ein Computer ihm dabei helfen?

Informatiker bezeichnen diese Frage als „Problem des Handlungsreisenden“, das zur Klasse der „NP-vollständigen“ Probleme gehört. Das heißt: Selbst die leistungsfähigsten Computer und die beste Software stoßen bei der Suche nach einer Lösung schnell an ihre Grenzen – und zwar was die Rechenzeit betrifft.

Die Komplexität des Problems resultiert daraus, dass die Zahl denkbarer Reisevarianten explosionsartig wächst, je mehr Zwischenziele der Nikolaus ansteuern will. Während bei fünf Stationen zwölf verschiedene Routen lauffähig wären, sind es bei zehn Stationen schon 181 440. Bei elf Stopps gibt es 1,8 Millionen und bei zwölf Stationen bereits 30 Millionen unterschiedliche Wegestrecken. Bis zu diesem Punkt könnte ein moderner Computer gerade noch die kürzeste Route berechnen. Aber schon bei 20 Stopps müsste das Programm viele Hundert Jahre lang laufen, um zu einer Lösung zu kommen. Abhilfe schaffen könnte eine neue Schlüsseltechnologie: der Quantencomputer.

Quantencomputer machen sich bestimmte quantenmechanische Effekte zunutze, wodurch sie bei speziellen Aufgaben einen enormen Geschwindigkeitsvorteil gegenüber klassischen Computern erreichen. So ist es etwa japanischen Forschern vergangenes Jahr gelungen, mithilfe eines speziellen Quanten-Chips ein Handlungsreisenden-Problem mit 22 Stationen zu lösen, für das ein Hochleistungsrechner mehr als tausend Jahre benötigt hätte.

Die Suche nach kürzesten Wegen hat dabei unzählige praktische Anwendungen von der Routenplanung für Paketdienste und Fluggesellschaften, der Optimierung von Lieferketten und Verkehrsflüssen, über das Design von Mikrochips, die Übertragung von Datenpaketen in Telekommunikationsnetzwerken bis hin zur Genom-Sequenzierung. Quantencomputer können ihre Überlegenheit aber nicht nur bei speziellen Optimierungsproblemen ausweisen, wie dem Problem des Handlungsreisenden oder bei Portfolio-Entscheidungen im Finanzsektor, sondern auch bei komplexen Simulationen (zum Beispiel von Molekülen für biochemische Anwendungen) und im Bereich der künstlichen Intelligenz (zum Beispiel zur Mustererkennung auf Basis neuronaler Netze).

In allen großen Wirtschaftsmächten – USA, China, Europa – hat man die strategische Bedeutung der neuen Quantentechnologien erkannt, zu denen neben Quantencomputern auch die Quantenkommunikation und die Quantensensorik zählen. Als Querschnittstechnologien beeinflussen sie derzeit viele Wirtschaftszweige, das kein Industrieland sich einen technologischen Rückstand leisten kann. McKinsey hat kürzlich die globale Wettbewerbsituation untersucht. Demnach haben in den vergangenen zehn Jahre sowohl die USA (rund 2,1 Milliarden) als auch Großbritannien (981 Millionen) und Kanada (658 Millionen Dollar) deutlich mehr in Quantentechnologien investiert als die EU (294 Millionen). Dabei spielen vor allem private Wagniskapitalinvestitionen eine Rolle: Sie machten mehr als die Hälfte der Gesamtinvestitionen aus. Seit 2019 sind die weltweiten Investitionen sprunghaft angestiegen.

Bei den staatlichen Mitteln, die in Zukunft für Quantentechnologien bereitgestellt werden sollen, liegt die EU mit angekündigten 7,2 Milliarden Dollar (davon rund drei Milliarden aus Deutschland) auf Rang zwei hinter China (15 Mrd.) und vor den USA (1,3 Mrd.). Bei den Patenten sind chinesische und japanische Erfinder Spitze. Bei wissenschaftlichen

THEMA
global challenges

Global Challenges ist eine Marke der DVM Medien. Das neue Layout möchte die Diskussion gesellschaftlicher Themen durch Veröffentlichungen anerkannter Experten vorantreiben.

HEUTE



Prof. Dr. Ann-Kristin Achleitner
Inhaberin des Lehrstuhls für Entrepreneurial Finance an der TU München

Weitere Autorinnen:
Günther Göttinger
Sigrun Gabriel
Jürgen Trittin
Prof. Jörg Rocholl PhD
Prof. Dr. Bert Rürup
Prof. Dr. Renate Schubert

Veröffentlichungen steht dagegen die EU auf Rang eins, knapp vor China. Am häufigsten zitiert werden allerdings amerikanische Veröffentlichungen. Über die größten Talentpools gemessen an Hochschulpublikationen in einschlägigen Disziplinen verfügen jedoch die EU und China.

Was die Anwendungsindustrien betrifft, sehen Experten das größte Disruptionspotenzial für Quantencomputer in der Chemie- und Pharmaindustrie. Dort könnten Simulationen molekularer Prozesse etwa die Medikamentenentwicklung drastisch beschleunigen. Ansonsten forciert vor allem die Telekommunikationsindustrie erste Anwendungen, unter anderem gefolgt von der Finanz-, Automobil- sowie der Luftfahrt- und Verteidigungsindustrie. Dagegen befinden sich Versicherungen, Transport-, Logistik- und Energieunternehmen häufig noch in vorwettbewerblichen Erprobungen. Im Gesundheitswesen und im öffentlichen Sektor sind bislang kaum Aktivitäten bekannt.

Worauf kommt es aus deutscher Perspektive jetzt an, um auf lange Sicht zu den Gewinnern der Quantenrevolution zu gehören? Ich sehe drei Schwerpunkte.

Fürstens: Die Anwenderindustrien sollten Quantencomputing bereits jetzt gezielt in ihr Geschäft integrieren. Schon heute lassen sich Quantenalgorithmen in der Praxis gewinnbringend einsetzen. Technisch basieren viele Anwendungsfälle noch auf Übergangslösungen, die Quanten- und konventionelle Rechensysteme kombinieren. Die Einstiegshürden sind dabei vergleichsweise niedrig. Die Leistungen können über die Cloud bezogen werden und erfordern von den Nutzern keine eigenen Hardware-Investitionen. Der frühe Einstieg in das Thema ist wichtig, um sich Wettbewerbsvorteile zu verschaffen, aber auch, um sich schon einmal voranzutun in der Quantenwelt, bevor es zu einem Durchbruch bei leistungsfähiger Quantenhardware kommt.

Zweitens: Die Bundesregierung sollte ihre Förderaktivitäten stärker koordinieren, um Synergien etwa zwischen den Programmen des Forschungs- und des Wirtschaftsministeriums gezielter zu nutzen. Diese Bündelung der Kräfte sollte im Schulterschluss mit dem Industriekonsortium „Qates“ erfolgen, das industrielle Anwendungen des Quantencomputings zur Marktreife bringen will. Überdenken sollte die Regierung ausnahmsweise auch den sehr breiten Ansatz der Technologieförderung. Aus heutiger Sicht ist zwar noch nicht sicher, welche Technologie Plattform sich bei Quantencomputern durchsetzen wird. Eine Förderung nach dem „disruptive innovation“-Prinzip birgt jedoch die Gefahr, dass sie bei keiner Technologie die kritische Masse für entscheidende Durchbrüche erreicht. Zudem brauchen wir eine Ausbildungsinitiative, die neben theoretischen Grundlagen vor allem auch anwendungsrelevante Kompetenzen adressiert.

Drittens: Deep-Tech-Start-ups spielen eine entscheidende Rolle für das Quanten-Ökosystem – auch wenn die mediale Aufmerksamkeit bislang vor allem den amerikanischen IT-Konzernen gilt. Wagniskapitalinvestitionen in diesem Bereich erfordern viel Geduld und tiefe Taschen. Der Staat kann das Engagement privater Investoren flankieren, sollte sich aber nicht allein auf die Rolle des Förderers beschränken. Mindestens genauso wichtig wäre es, dass er selbst zu den ersten Kunden der neuen Schlüsseltechnologien gebiert und damit der Quantenrevolution den entscheidenden Schub gibt.

Wir haben in Deutschland und Europa jetzt die einmalige Chance, unsere starke Position in der Forschung auch in wirtschaftliche Stärke und einen nachhaltigen Wettbewerbsvorteil umzusetzen. Wir sollten sie nicht verpassen.

„But even at 20 stops, the program would have to run for many hundreds of years to arrive at a solution. The remedy could be a new key technology: quantum computing.“

$$20! = 2.43 \cdot 10^{18}$$

The algorithm described in 1962 by Bellmann / Held and Karp uses dynamic programming to compute the best solution in

$$O(2^n n^2) = 2^{20} \cdot 400 = 419 \text{ M Steps.}$$

This can be done in under 1 s on a Laptop.

https://en.wikipedia.org/wiki/Held%E2%80%93Karp_algorithm

<https://www.handelsblatt.com/meinung/gastbeitraege/gastkommentar-die-chancen-der-quantenrevolution-/27825902.html> Handelsblatt 24.11.2021

<https://www.tagesspiegel.de/politik/alles-auf-quantencomputer-warum-es-lohnt-auf-die-schlüsseltechnologie-zu-setzen/27829098.html> Tagesspiegel 25.11.2021

Name	System	CPU Cores	HT	GHz	RAM Size	M.Msteps/s
RP1	Raspberry-Pi 1B Rev 2 1176JZF-S ARMv6 (v6l)	1	1	1	0.70 512 MB	23 126@10
RP3	Raspberry-Pi 3B Cortex-A53 ARMv7 rev 4 (v7l)	1	4	1	1.20 2 GB	25 553@14
RP4	Raspberry-Pi 4B Cortex-A72 ARMv7 rev 3 (v7l)	1	4	1	1.80 4 GB	25 1,091@18
JN2					GB	27 1,522@16
OPT					GB	30 9,019@20
DGX					GB	26 19,228@25
					GB	32 18,000@25
					GB	28 36,443@28
SRV	Intel Xeon E7-8880 v4 [4] (broadwell)	4	22	2	2.20-3.50 2 TB	32 30,108@26
ZCG	Intel Xeon E5-2640 v4 [3] (broadwell) + A100 GB	1	10	2	2.40-3.40 512 GB	
					32 GB	29 53,592@29
ZC2	AMD EPYC 7542 [1] (zver2)	2	64	2	2.90-3.40 1 TB	32 14,998@25

**Warning: Just an example.
This is a stupid wrong way
to solve TSPs !!!**

Raspberry Pi 1
can do 23 cities

4.4 T steps
249 GiB
250 s

fastest

Growth Factors in 63 years:

cities = 2,600

var. = 7,3 mio

Trying to solve even bigger TSP-problems, linear programs with 2,162,098,780,185 (> 2 trillion) variables are solved to optimality

Year	Authors	# cities	# variables
1954	DFJ	42/49	820/1146
1977	G	120	7140
1987	PR	532	141,246
1988	GH	666	221,445
1991	PR	2,392	2,859,636
1992	ABCC	3,038	4,613,203
1994	ABCC	7,397	27,354,106
1998	ABCC	13,509	91,239,786
2001	ABCC	15,112	114,178,716
2004	ABCC	24,978	311,937,753
2005	ABCCEG	33,810	571,541,145
2006	ABCCEG	85,900	3,689,620,050
2017	ABCCGH	109,399	5,984,015,901

Quantum computers may be more of an imminent threat than AI

Quantum computers are straight out of science fiction. Take the “**traveling salesman problem**,” where a salesperson has to visit a specific set of cities, each only once, and return to the first city by the most efficient route possible. As the number of cities increases, the problem becomes exponentially complex. **It would take a laptop computer 1,000 years to compute the most efficient route between 22 cities**, for example. **A quantum computer could do this within minutes, possibly seconds.**

https://www.washingtonpost.com/news/innovations/wp/2018/02/05/quantum-computers-may-be-more-of-an-imminent-threat-than-ai/?noredirect=on&utm_term=.123bc8dcf1d2

```
excession% time ~/concorde/buildcpx/TSP/concorde -k22 -r100000
Using KP (SYN-version 2.03.0:Unversioned directory, built Sep 25 2017-08:44:42)
/Users/bico/concorde/buildcpx/TSP/concorde -k22 -r100000
Host: excession.local Current process id: 13789
Using random seed 1518466376
Random 22 point set, gridsize = 100000
Set initial upperbound to 375020 (from tour)
LP Value 1: 356195.500000 (0.00 seconds)
LP Value 2: 375020.000000 (0.00 seconds)
New lower bound: 375020.000000
Final lower bound 375020.000000, upper bound 375020.000000
Cutoff less than ZERO, do not no elim
Exact lower bound: 375020.000000
DIFF: 0.000000
Final LP has 26 rows, 60 columns, 142 nonzeros
Optimal Solution: 375020.00
Number of bbnodes: 1
Total Running Time: 0.00 (seconds)
0.005u 0.002s 0:00.00 0.01
```



Bill Cook
@wjcook

Folgen

To clarify, computing an optimal TSP route on 22 points takes 0.005 seconds on my iMac, not the 1000 years reported in **@washingtonpost** An error factor of 6 trillion Like reporting the US National Debt is \$4. [washingtonpost.com/news/innovations/](https://www.washingtonpost.com/news/innovations/) ...

The Traveling Salesmen Problem

Binary
variables

Given a graph $G = (V, E, c)$ and binary variables $x_e \in \{0,1\}$ for all $e \in E$ we can model the TSP as a Mixed Integer Linear Program as follows:

$$\min \sum_{e \in E} c_e x_e \quad \text{subject to} \quad \sum_{e \in \delta(S)} x_e \geq 2 \quad \text{for all } S \subset N, S \neq \emptyset$$

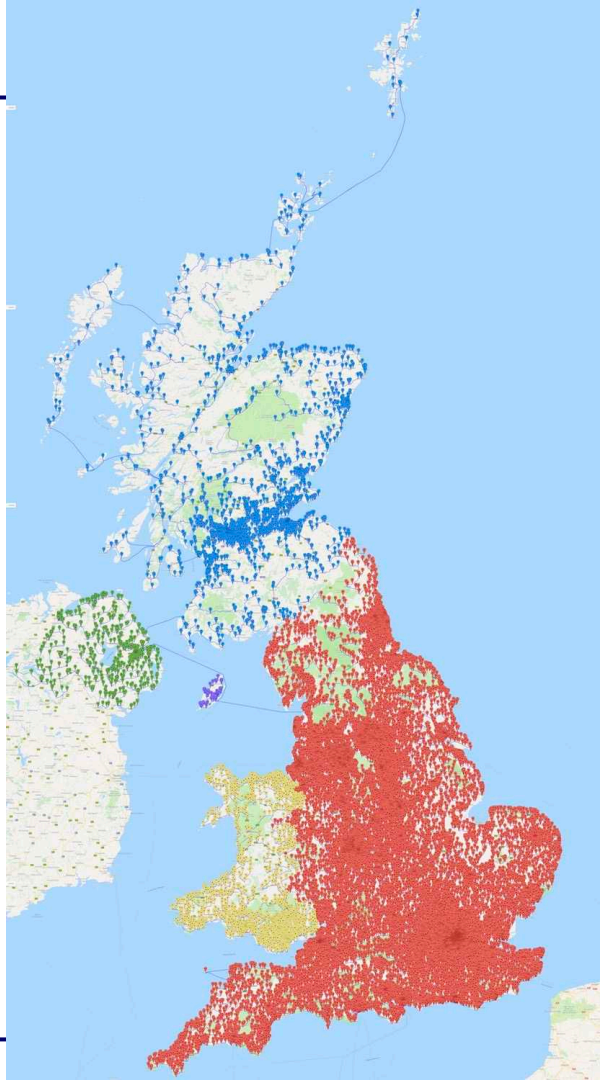
Small
coefficients

with $\delta(S) := \{e = (i, j) \in E \mid i \in S, j \in N \setminus S\}$.

Every proper subset (starting from a single node) has exactly two edges connecting it to the rest of the graph.

Proven Optimal UK pub tours
visiting 49,687 pubs:

<http://www.math.uwaterloo.ca/tsp/uk>



- ▷ The TSP is NP-complete.
- ▷ There are $n!$ possible permutations, we could try.
- ▷ Every such permutation is a feasible solution to the problem.
- ▷ **QC just (non deterministically) computes some solution.**
- ▷ It is not difficult to find a “good” solution within a few percent of the optimum. If you are looking for a quick and very good solutions, look here

K. Helsgaun, Technical Report, Roskilde University, 2017

[An Extension of the Lin-Kernighan-Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems.](http://webhotel4.ruc.dk/~keld/research/LKH-3)

<http://webhotel4.ruc.dk/~keld/research/LKH-3> (Also solves 39 variants.)

- ▷ **The difficulty is proving optimality! QC is not good at this.**
- ▷ Worst case complexity is better than $O(2^{n-1} \cdot n^2)$ and not $O(n!)$.
- ▷ The by far best software for this is CONCORD using LP-based Branch&Cut
<https://www.math.uwaterloo.ca/tsp/concorde.html>
Needed typically 3 min to solve 1.000 city TSP to optimality in 2003 !
- ▷ If you are interested in the topic: read THIS BOOK ----->

Princeton Series in APPLIED MATHEMATICS

The Traveling Salesman Problem

A Computational Study



David L. Applegate,
Robert E. Bixby, Vašek Chvátal,
and William J. Cook

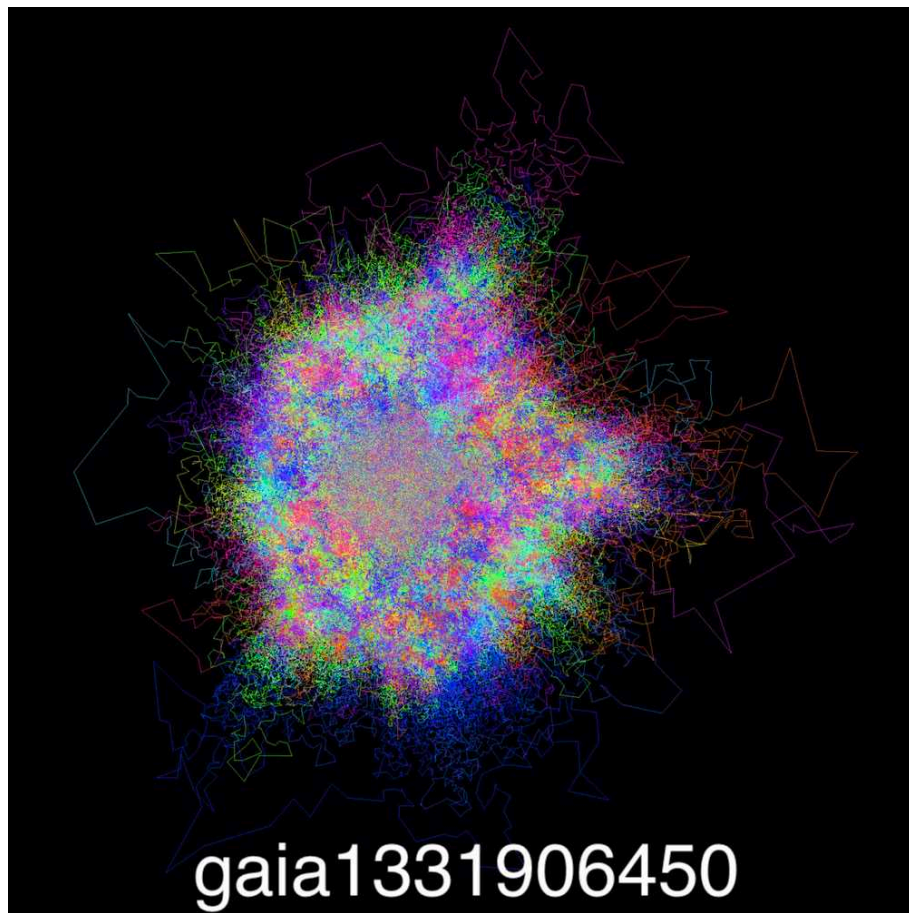
Tour length: 4,961,937,077 parsecs,
or about 16.2 billion light years

The tour is at most 1.0038 times the length of a
shortest-possible route.

The 1.0038 factor is nice, but it's not nearly as
strong as the quality guarantees we have for the
smaller star-TSP instances. The tour itself is likely
much better than the gap indicates, and we are
improving it nearly every day. The LP bound is the
culprit.

<https://www.math.uwaterloo.ca/tsp/star>

Note: Cook, Held, Helsgaun just won the Amazon Last Mile Routing
Research Challenge decisively, using no NN or whatever.



What is the complexity class of Exercise 02-2, the Collatz/hailstone computation?

What is the worst-case runtime $\mathcal{O}$?

How is the runtime for $a_i > \epsilon = 2^{64}$ fixed?

=> In the fixed case the algorithm always runs a fixed number of steps, therefore, runtime is $\mathcal{O}(1)$.

How is the runtime for an arbitrary ϵ ? What do you think?

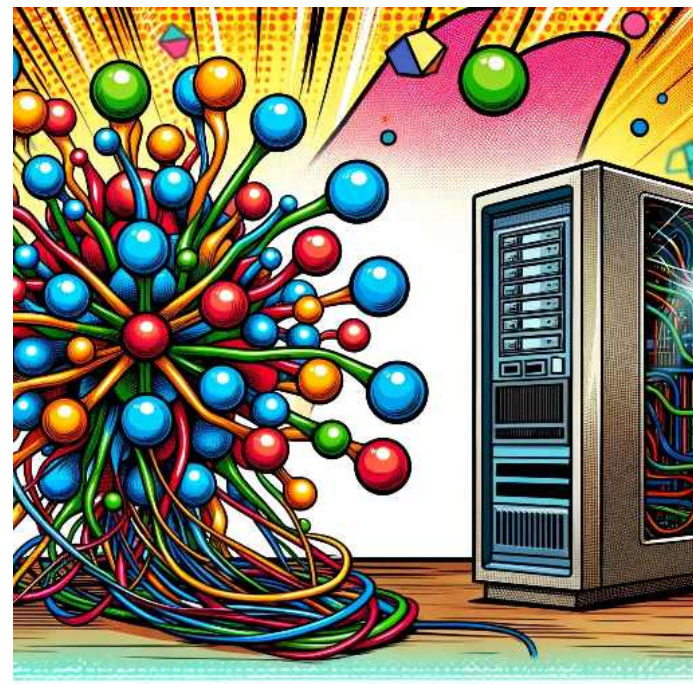
Testing all numbers up to ϵ takes at most $\mathcal{O}(\epsilon)$ steps.

We start at 3 and test all numbers up to ϵ in order. Each number we encounter in any sequence we mark as visited. Whenever we hit an already visited number we stop and try the new unvisited number as start. A visited number either has already been shown to run down to 1 or is part of a loop that never reaches ϵ . Latest when we reach ϵ we will encounter a number in the sequence $> \epsilon$. The work to be done at any number is constant. Therefore, we will finish after at most $\mathcal{O}(\epsilon)$ steps.

However, note the input length is $\log_2 \epsilon$ and therefore we have $\mathcal{O}(2^\epsilon)$ step in relation to the input length. Every bit the input gets longer, doubles our effort.

Benchmarking can provide insights into realistic algorithmic performance when purely analytical methods fail, [...] In particular, theoretical complexity analysis typically pertains to general problem classes and considers mostly asymptotic scaling behavior or worst-case scenarios.

In fact, **theoretical worst-case bounds, often fail to describe the difficulties of solving real-world applications that focus on a specific problem instance of practical use, nor do they faithfully describe how well practically relevant problems can be solved for fixed size and data.**



A problem class being NP-hard says nothing about how difficult it is to solve a particular instance from this class.

- ▶ Comparing different hardware running the same software (source code).
Examples: LINPACK <https://www.top500.org>, SPEC <https://www.spec.org>
- ▶ Comparing different software (source/binaries) on the same hardware.
Examples: MIPLIB <https://miplib.zib.de>
- ▶ Tracking the progress in a field over time (multiple hardware/software).
Progress in Mathematical Programming Solvers from 2001 to 2020,
K., Berthold, Pedersen, Vanaret, EJCO, doi: 10.1016/j.ejco.2022.100031
- ▶ Comparing the state of the art in a field (multiple hardware/software)
Computer Olympiad https://en.wikipedia.org/wiki/Computer_Olympiad
Computer World Chess Championship
https://en.wikipedia.org/wiki/World_Computer_Chess_Championship
- ▶ *We do not know now future Quantum Hard- and Software. **Usually, there are many way to model a given problem.** Therefore, we need **model independent benchmarks**, where a problem and the accompanying data is given, but no choice is made how it should be solved. However, the problem and expected solution must be precisely defined, such that the solutions can be compared.*



HW Developer

Given: HW, Algorithm, Problem

- Model dependent benchmarks
- Find the best HW-setting to run a given algorithm

Goal:

- Improve HW to give better performance

Algorithm Developer

Given: Algorithm, Problem

- Model dependent and/or independent benchmarks
- Find the best settings / implementation to run an algorithm on some HW

Goal:

- Get the best possible performance for a given algorithm

Application Developer

Given: Problem

- Model independent benchmarks
- Find the best HW/SW combo for a given problem
- Compare vs state-of-the-art

Goal:

- Demonstrate advantage


```
use std::time::Instant;

let start_time = Instant::now();
// Do something here
let duration    = start_time.elapsed();

println!("Time : {:?}", duration);
println!("Time : {:5}", duration.as_millis());
```

Formatted printing: `{:12.3e}`

See <https://doc.rust-lang.org/std/fmt>

LP Progress: 1988 — Present

(Operations Research, Jan 2000 — 15)

- Algorithms (*machine independent*):

Primal *versus* Primal/Dual/Barrier

3300x

- Machine (workstations → PCs):

1600x

- Algorithm × Machine **5 300 000x**

(2 months/5300000 ≈ 1 second)

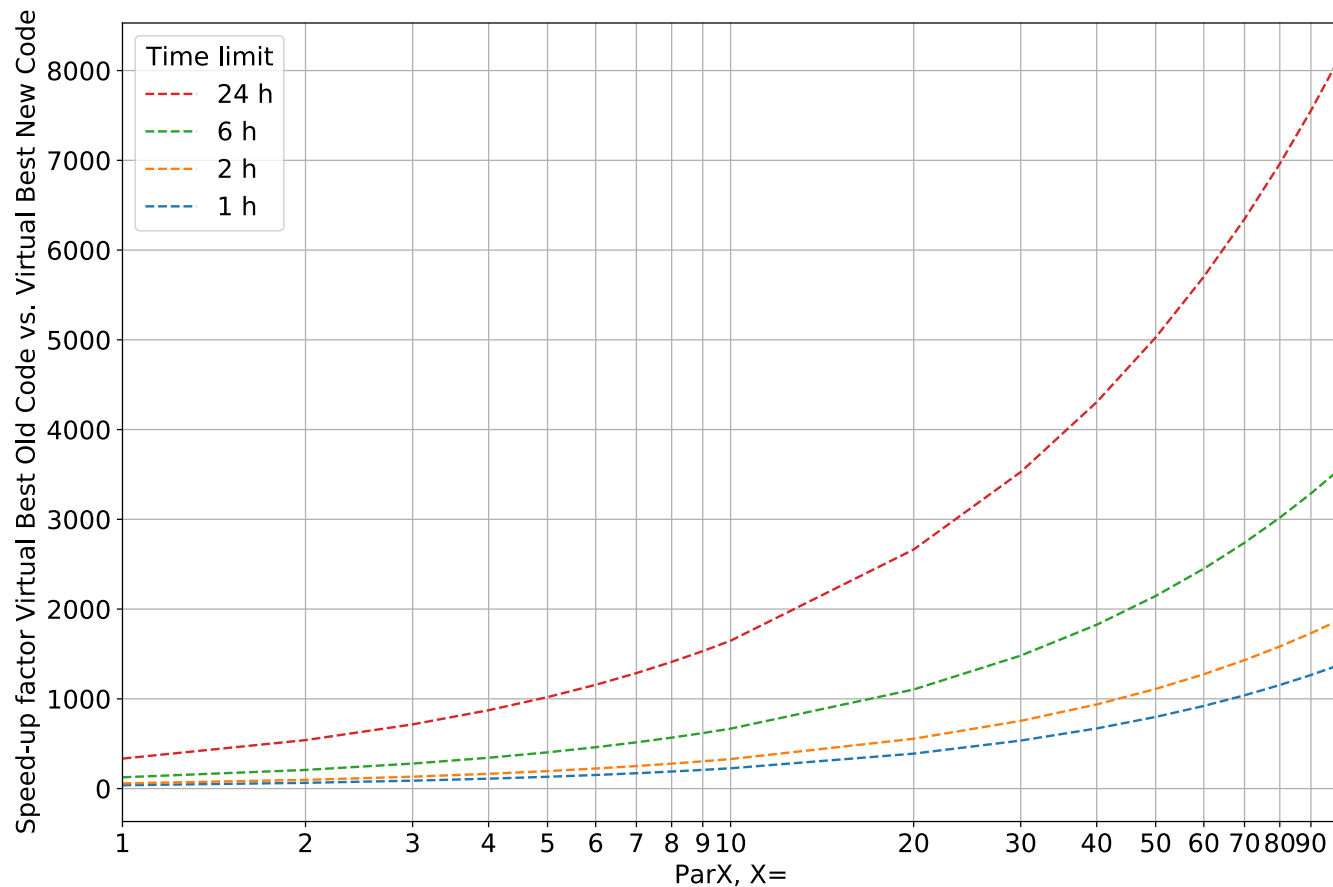
How long do we want to wait?

We observe:

- ▷ We have to set a time limit.
- ▷ Some instances will not solve to optimality within the time limit on some machine/solver combi.
- ▷ Some instances might solve too fast to be measured accurately on some machine/solver combi.
- ▷ Measuring CPU time does not work in the parallel case and wall time might have high variability.

Questions that have to be answered:

- ▷ Which machine to use?
- ▷ Which solver to use?
- ▷ Which instances to use?
- ▷ Which time limit to use?
- ▷ What to do if the time limit is hit?
- ▷ Which amount of memory and how many cores?
- ▷ But wondersolver can do much better on cursed.lp if you set parameter abracadabra=42
- ▷ Can we do the experiment in such a way that we can explain the result?



```
const THRESHOLD : u128 = 1 << 64;

pub fn collatz_u128(i :u64) -> u64 {
    debug_assert!(i % 2 == 1);

    let mut n = (3 * i + 1) / 2;

    while n > i {
        if n % 2 == 1 {
            let mut k : u128 = n as u128;
            k *= 3;
            k += 1;

            if k > THRESHOLD {
                return i;
            }
            n = k as u64;
        }
        n /= 2;
    }
    u64::MAX
}
```

```
pub fn collatz_u128_cached(i :u64, visited : &mut [bool]) -> u64 {
    let mut n = (3 * i + 1) / 2;

    while n > i {
        if n % 2 == 1 {
            if n < CACHE_MAX {
                if visited[n as usize / 2] {
                    return u64::MAX;
                }
                visited[n as usize / 2] = true;
            }
            let mut k : u128 = n as u128;
            k *= 3;
            k += 1;
            if k > THRESHOLD {
                return i;
            }
            n = k as u64;
        }
        n /= 2;
    }
    u64::MAX
}
```

```
pub fn collatz_tryfrom(i :u64) -> u64 {
    debug_assert!(i % 2 == 1);

    let mut n = (3 * i + 1) / 2;

    while n > i {
        if n % 2 == 1 {
            let mut k : u128 = n as u128;
            k *= 3;
            k += 1;
            // ??? overflow happens at 2^64
            if let Ok(v) = u64::try_from(k) {
                n = v;
            } else {
                return i;
            }
        }
        n /= 2;
    }
    u64::MAX
}
```

```
pub fn collatz_overflow(i :u64) -> u64 {
    debug_assert!(i % 2 == 1);

    let mut n = (3 * i + 1) / 2;

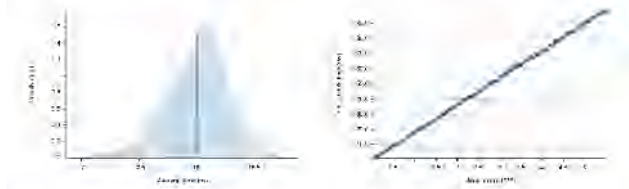
    while n > i {
        if n % 2 == 1 {
            // compute 3n + 1, catching overflow
            let (tripled, ovf1) = n.overflowing_mul(3);
            let (n_next, ovf2) = tripled.overflowing_add(1);
            // ??? overflow happens at 2^64
            if ovf1 || ovf2 {
                return i;
            }
            n = n_next;
        }
        n /= 2;
    }
    u64::MAX
}
```


Apple M4 Max

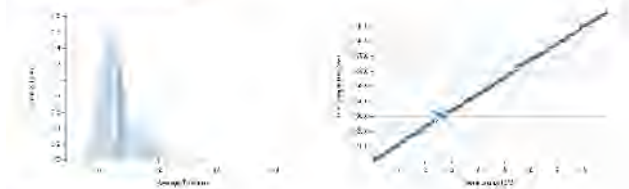


This chart shows the relationship between function evaluations and function error. The minimum error indicates the probability that a measurement of the given function parameter would have a deviation of 10^{-6} or less.

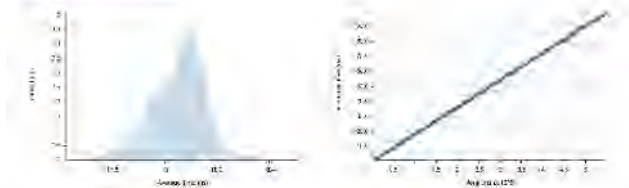
Halton search implementation on Apple M4 Max



Halton search implementation on Apple M4 Max



Halton search implementation on Apple M4 Max

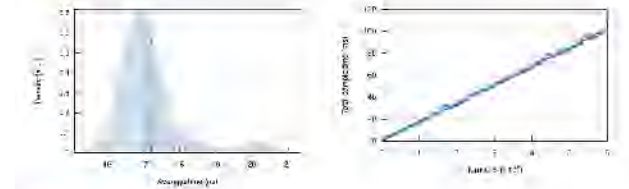


AMD Ryzen Threadripper PRO 7955WX

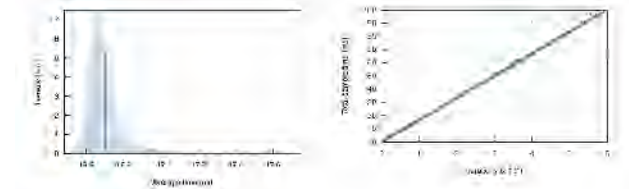


This chart shows the relationship between function evaluations and function error. The minimum error indicates the probability that a measurement of the given function parameter would have a deviation of 10^{-6} or less.

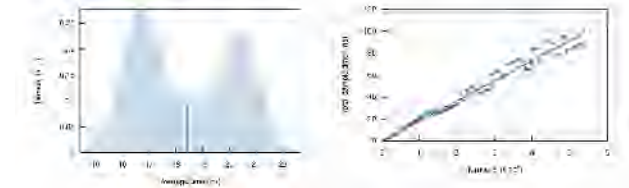
Halton search implementation on AMD Ryzen Threadripper PRO 7955WX



Halton search implementation on AMD Ryzen Threadripper PRO 7955WX



Halton search implementation on AMD Ryzen Threadripper PRO 7955WX



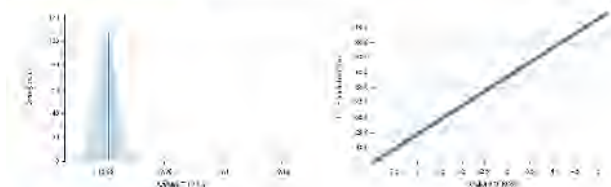
NVIDIA GraceHopper100

mlu store search implementation: Vector dot



This plot shows the distribution of the number of iterations for the mlu store search implementation on NVIDIA GraceHopper100. The x-axis represents the number of iterations, and the y-axis represents the density of the distribution. The distribution is centered around 50 iterations.

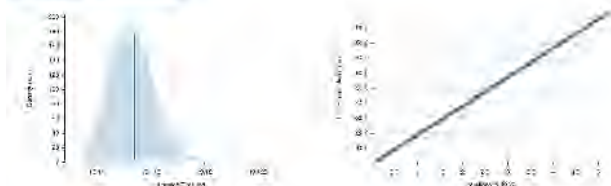
Iterations per second (iterations/second)



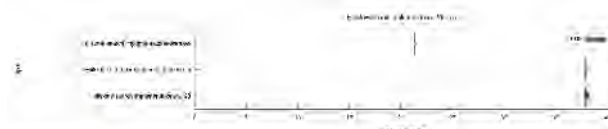
Iterations per second (iterations/second)



Iterations per second (iterations/second)

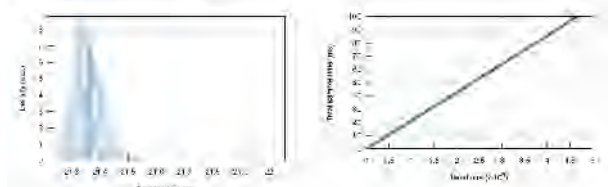


Intel Core i7-9700K

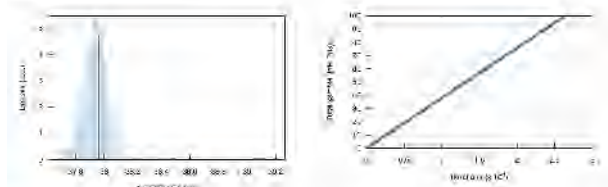


This plot shows the distribution of the number of iterations for the Intel Core i7-9700K. The x-axis represents the number of iterations, and the y-axis represents the density of the distribution. The distribution is centered around 50 iterations.

Iterations per second (iterations/second)



Iterations per second (iterations/second)



Iterations per second (iterations/second)



```
const CACHE_SIZE : usize = 1 << 33;
const HAIL_MAX    : u64   = u64::MAX / 3;
```

```
fn hailstone_seq_u128() -> u64 {
    for i in (5 .. HAIL_MAX).step_by(2) {
        if collatz_u128(i) < u64::MAX {
            return i;
        }
    }
    unreachable!("Cannot happen!");
}
```

```
fn hailstone_seq_u128_3of4() -> u64 {
    for i in (7 .. HAIL_MAX).step_by(8) {
        if collatz_u128(i) < u64::MAX {
            return i;
        }
        if collatz_u128(i + 2) < u64::MAX {
            return i;
        }
        if collatz_u128(i + 4) < u64::MAX {
            return i;
        }
    }
    unreachable!("Cannot happen!");
}
```

```
fn hailstone_seq_u128_cache() -> u64 {
    let mut visited = vec![false; CACHE_SIZE];

    for i in (5 .. HAIL_MAX).step_by(2) {
        if collatz_u128_cached(i, &mut visited) < u64::MAX {
            return i;
        }
    }
    unreachable!("Cannot happen!");
}
```

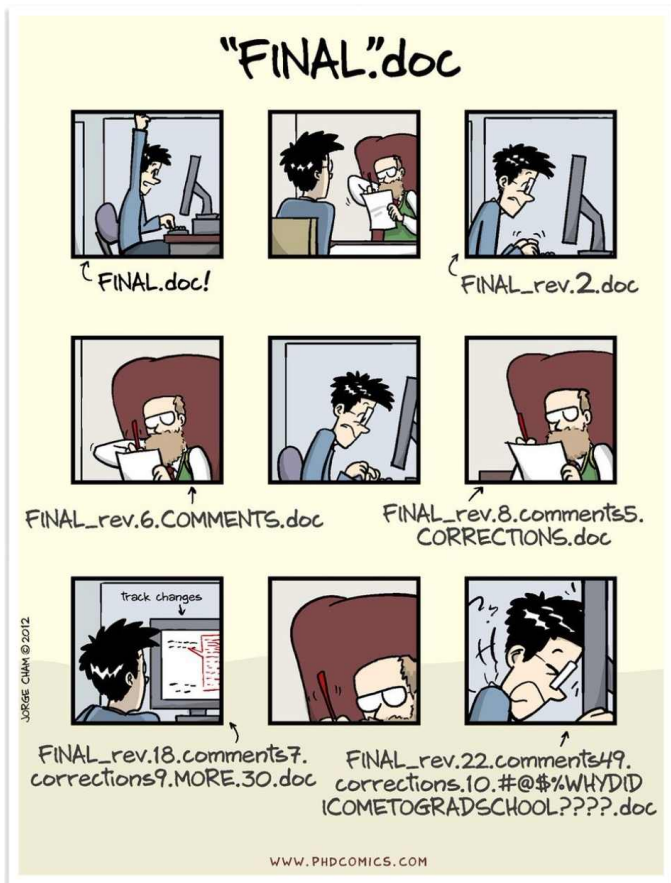
```
fn hailstone_par_u128() -> u64 {
    let chunk_size = rayon::current_num_threads() * 8 * 1024 * 1024;

    for i in (5 .. HAIL_MAX).step_by(2 * chunk_size) {
        let start = (0..chunk_size)
            .into_par_iter()
            .map(|k| { collatz_u128(i + 2 * k as u64) })
            .reduce(|| u64::MAX, u64::min);

        if start < u64::MAX {
            return start;
        }
    }
    unreachable!("Cannot happen!");
}
```


Student presentation

Introduction to git



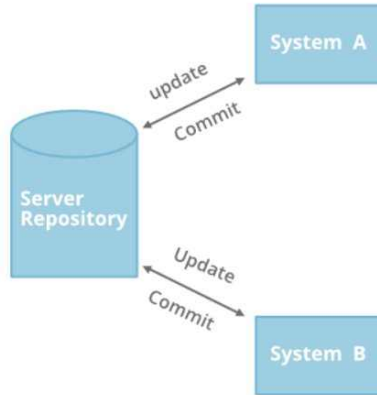
It helps you!

- Transparent history of all changes
- Moving back and forth in time
- Everything is easily traceable and reversible

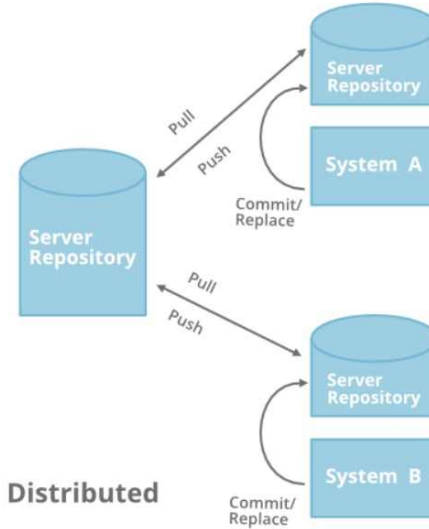
Good scientific practices:

- Reproducibility and traceability
- Enables collaboration

- Central Repository
- Single development
- Need a network connection
- A single point of failure



Centralized



Distributed

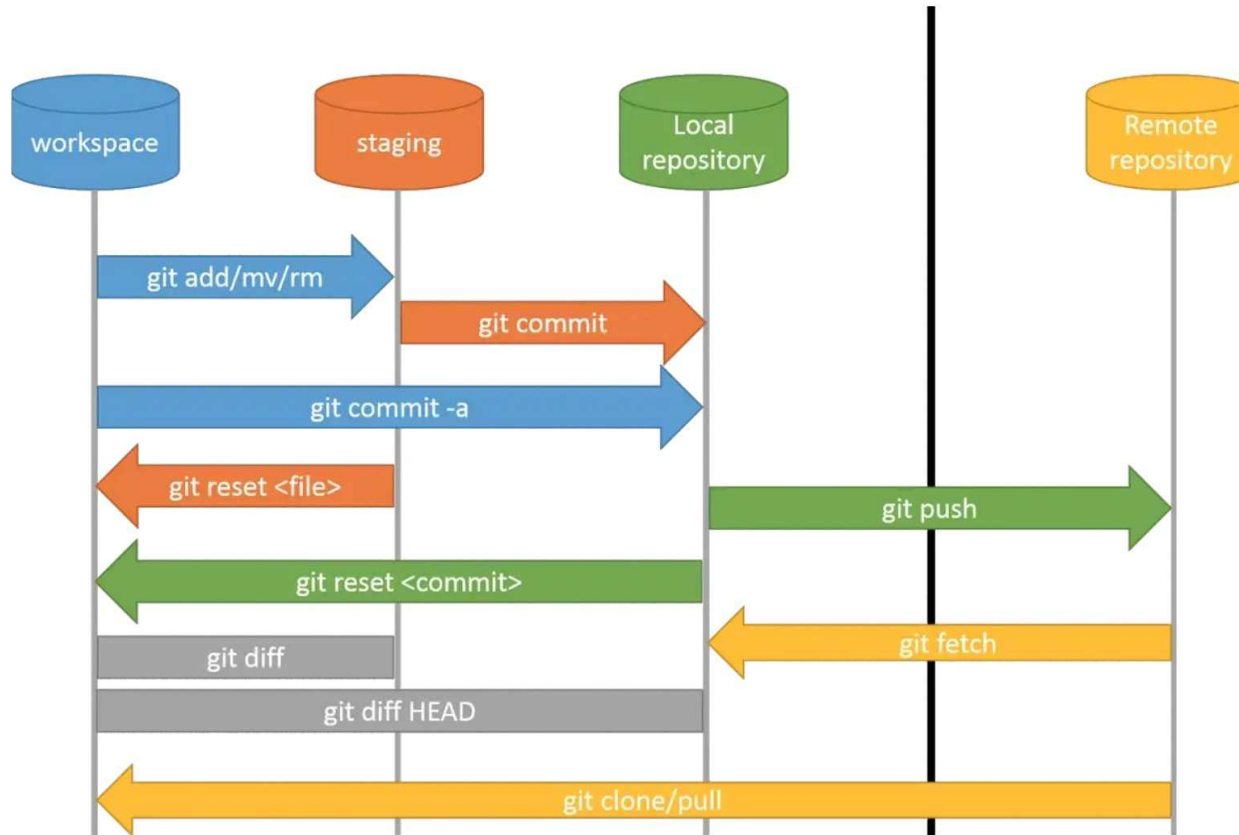
- Local Repositories
- Distributed development
- Flexibility
- Redundancy and Backup

<https://www.geeksforgeeks.org/centralized-vs-distributed-version-control-which-one-should-we-choose/>

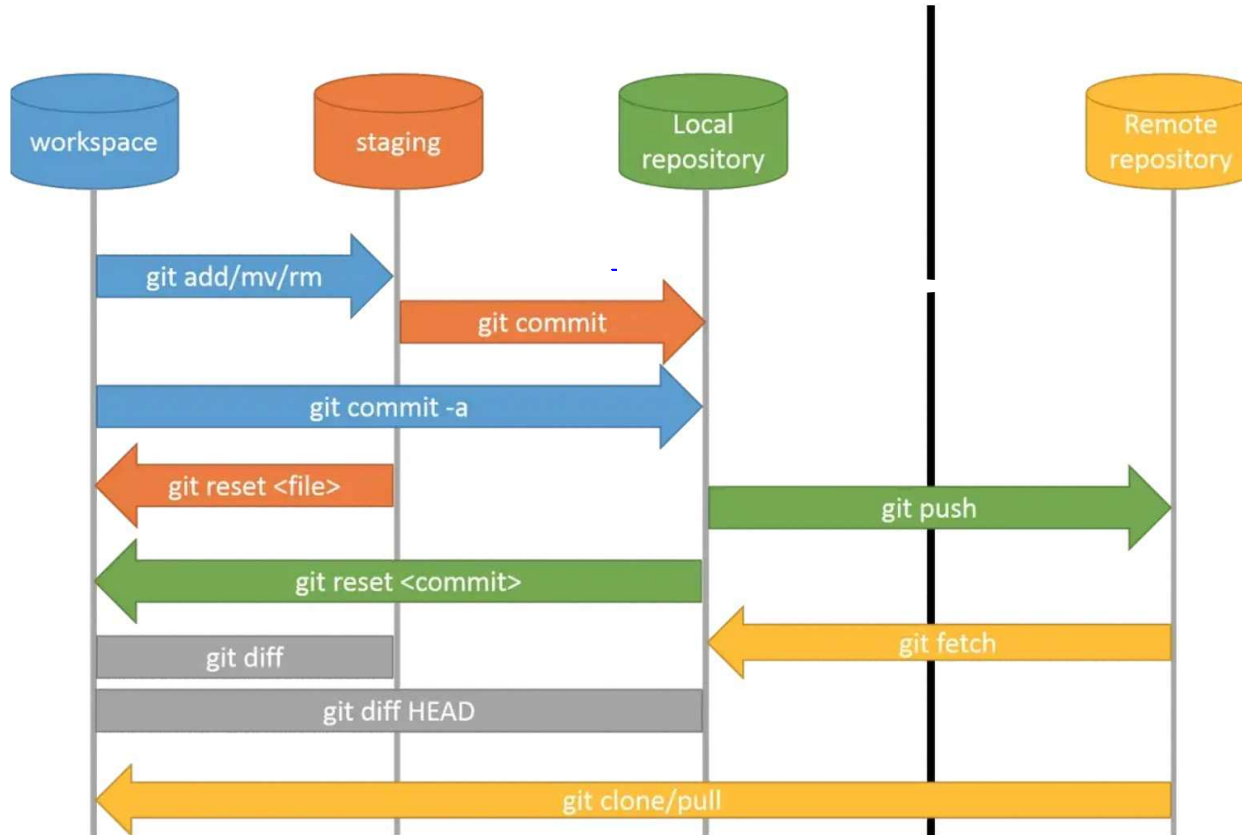
- ▷ [Git Homepage](#)
- ▷ [Download git](#)
- ▷ [Link to install git](#)
- ▷ Working it with the terminal: [Link of basic commands](#) on Linux/Mac OS
- ▷ [Using Git source control in VS Code](#)
- ▷ [Introduction to Git in VS Code](#)

- **Repository** (or repo): local or remote
- **Commit**: a unique identifier with a message of change
- **Branch**: making changes without affecting the main codebase
- **Remote**: GitHub, GitLab
- **Clone**: downloading entire repo
- **Pull**: updating a local repo with changes from a remote repo
- **Push**: sending local commits to a remote repo
- **Merge**: integrating work from one branch into another
- **Pull Request**: proposing change to a repo
- **Conflict**: two or more branches have made a changes to the same part of a life.

- `$ git config` to identify yourself
- `$ git init` to initialize an empty local git repo
- `$ git status` to display information of working directory and staging area
- `$ git add` to add changes to the staging area
- `$ git commit` to commit staged changes to the local repo
- `$ git diff` to display changes to your tracked files
- `$ git log` to display history of your commits
- `$ git branch` to store different version of your projects
- `$ git checkout` to switch between existing branches
- `$ git merge` to merge changes in checked out branch
- `$ git clone` to download a remote repository
- `$ git push` to create a new branch in the remote repo and update the remote branch
- `$ git pull` to update a branch with the new version from the remote repo after checking out

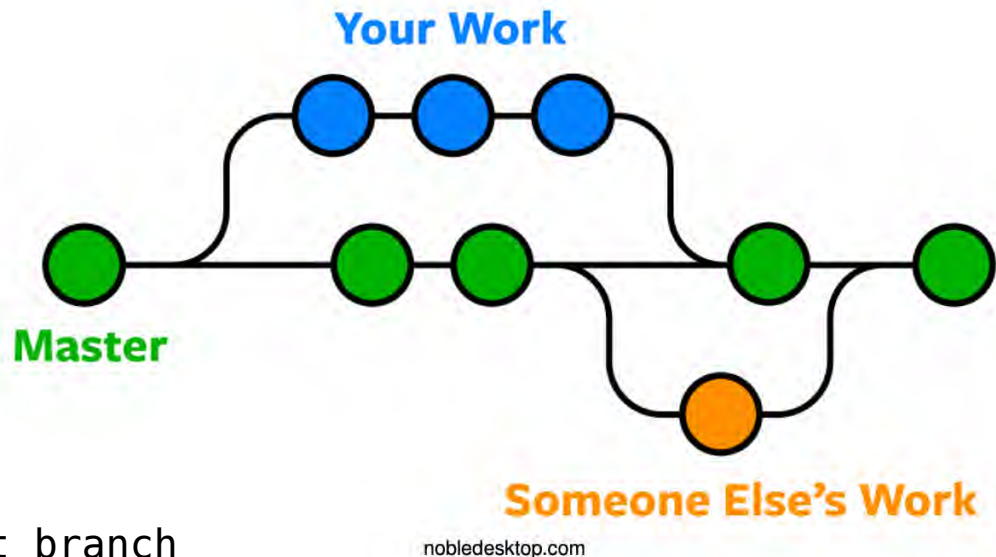


<https://medium.com/@samudithahasantha12>



<https://medium.com/@samudithahasantha12>

- Store different versions of your project
- Parallel development
- Main branch = master (by default)



Commands:

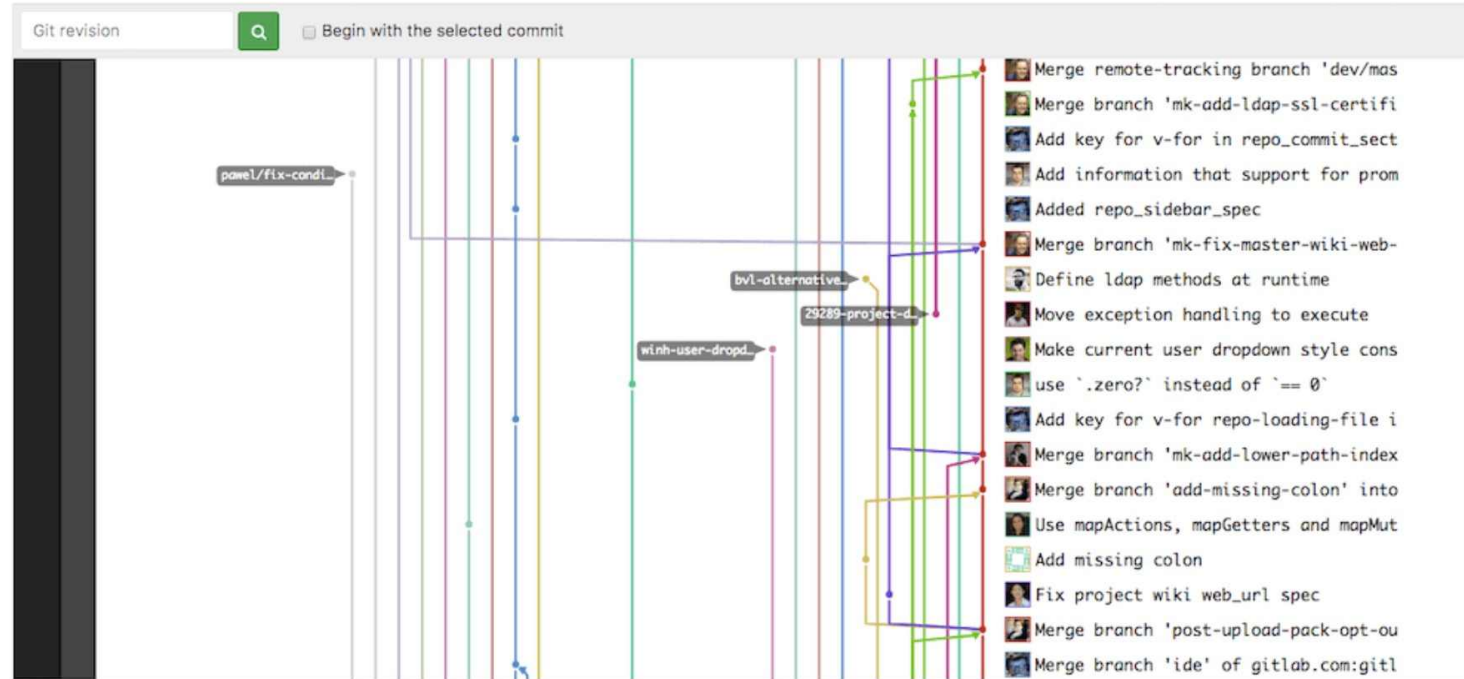
- List all branches of local repository: `$ git branch`
- Create a new branch: `$ git branch <branch_name>`
- Delete a branch: `$ git branch -d <branch_name>`
- Move to another branch: `git checkout <branch_name>`
- Create and check out a new branch: `git checkout -b <branch_name>`

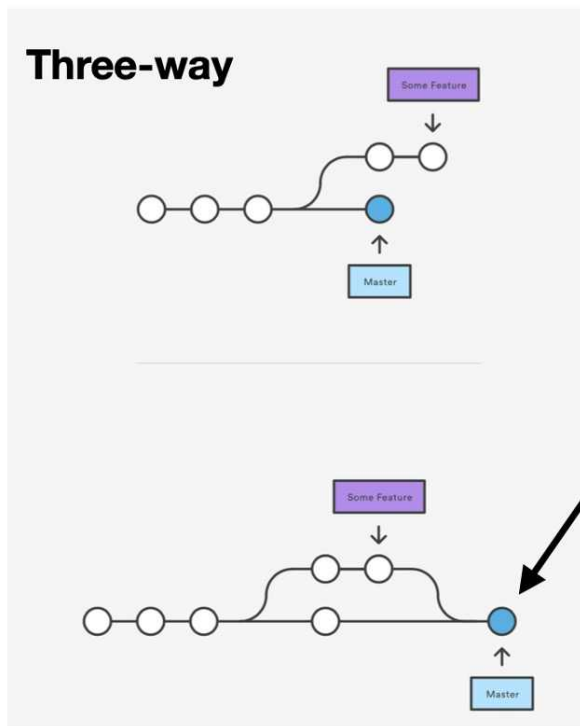
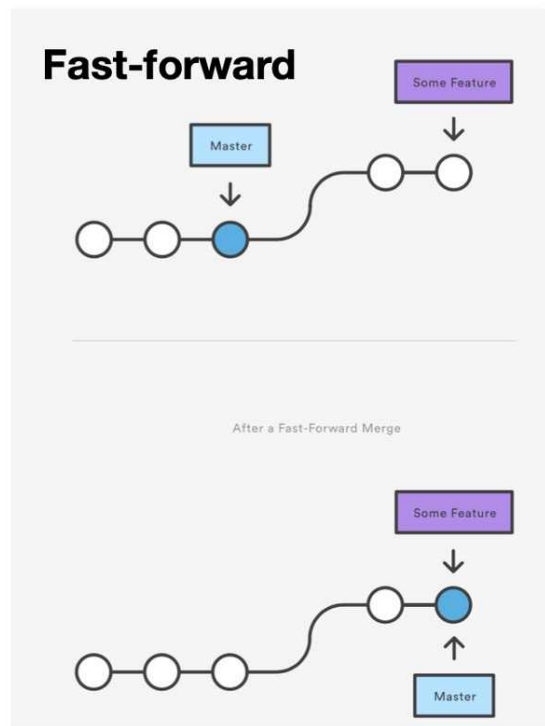
git branches (when it gets intense)

GitLab / GitLab.org / GitLab Community Edition
Repository / Graph

master

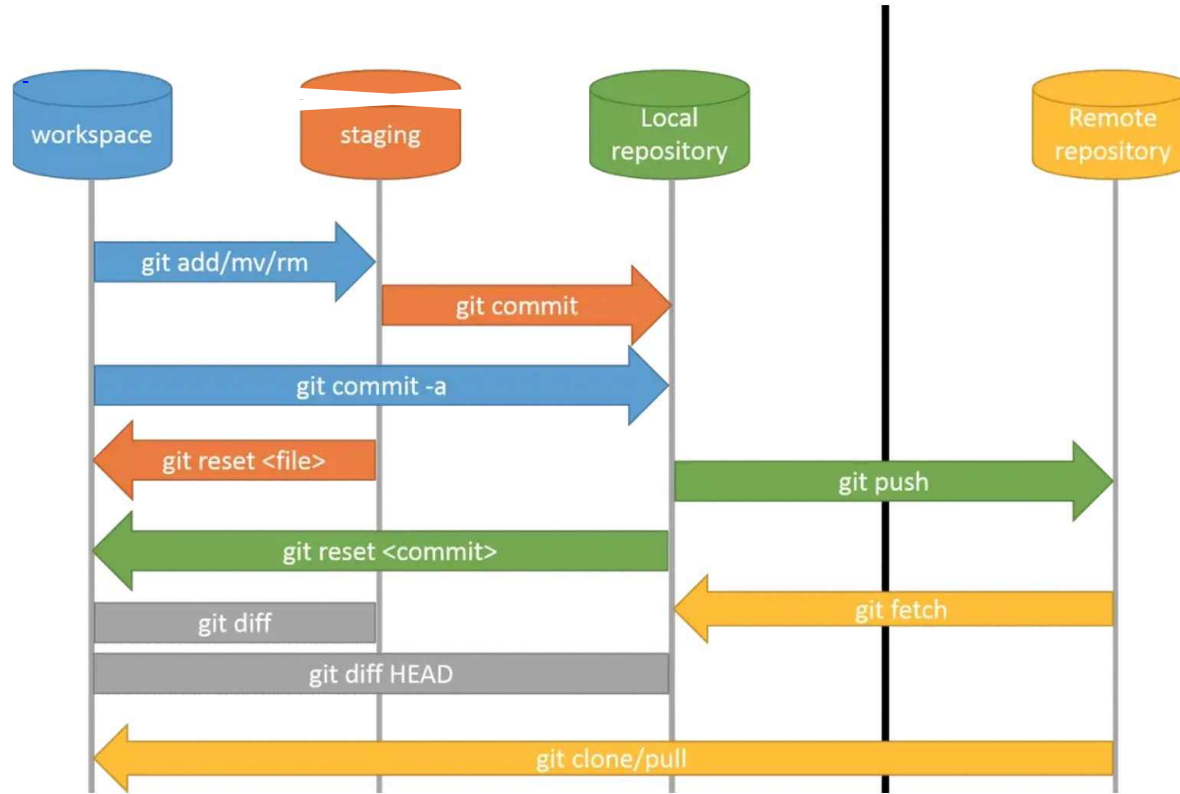
You can move around the graph by using the arrow keys.





Only three-way
merges have
merge
commits
and potential
merge conflicts

**If there are conflicting changes between the source and target branches,
Git will not be able to automatically reconcile them.**



<https://medium.com/@samudithahasantha12>

Clone (download) a remote repository

```
$ git clone <link_to_repository>
```

Creates directory with project name in current directory

Remote repository is (by default) referred to as `origin`

You can also clone on the same machine locally

See all remote branches of repository

```
$ git branch -r
```

Branches that are in remote are prefixed by `origin/` then the `<branch_name>`

Checkout a branch of the repository (as usual)

```
$ git checkout <branch_name>
```

Do not use the `origin/` prefix here

Update a branch with the new version from the remote repository

```
$ git pull
```

... while the branch is checked out

Changes your working directory

Create a new branch in the remote repository

```
$ git push -u origin <branch_name>
```

from the currently checked out branch

Update the remote branch from the local branch afterwards

```
$ git push
```

Only changes that are committed are pushed

If the remote and local history diverge (e.g. forgot to pull before committing) pushes will be rejected

 **cvanaret** / **Uno** Public

 Unpin  Unwatch 5  Fork 2  Starred 43

[Code](#) [Issues](#) 1 [Pull requests](#) [Discussions](#) [Actions](#) [Projects](#) [Security](#) [Insights](#) [Settings](#)

 main  3 branches  0 tags [Go to file](#) [Add file](#) [Code](#)

Charlie Vanaret Removed proximal term in feasibility problem 7d2141f 4 days ago 871 commits

	docs	Fixed performance profile legend	3 months ago
	examples	Added examples/ directory (AMPL .mod files + corresponding .nl files)	4 months ago
	uno	Removed proximal term in feasibility problem	4 days ago
	unotest	Cleaned up CMakeLists	26 days ago
	CITATION.cff	CITATION.cff file	4 months ago
	CMakeLists.txt	Cleaned up CMakeLists	26 days ago
	LICENSE	Updated date in license	10 months ago
	README.md	More compact README	24 days ago
	uno.options	Moved least_square_multiplier_max_norm (used in IPM) to option file	3 months ago
	uno_ampl-completion.bash	Renamed l1MeritFunction into MeritFunction (norm determined by cons...	4 months ago

About 

Uno: a modular open-source solver for unifying nonlinear optimization

[uno-unifying-nonlinear-optimization.re...](#)

cplusplus

math

optimization

mathematics

maths

gradient-descent

optimization-methods

numerical-optimization

nonlinear-programming-algorithms

nonlinear-optimization

optimization-algorithms

nonlinear-programming

cplusplus-17

newton-method

nlp-solver

uno-solver

unosolver

 Readme

 MIT license

 Cite this repository

 43 stars

github.com/cvanaret/Uno

We can prove, the program will

1. Terminate,
2. not raise exceptions, i.e., no overflows, division by zero, out of bounds, etc.,
3. produce a correct result upon termination.

If you cannot do (3) the best alternative is to produce a certificate, which allows to check the correctness of the result with less effort than computing it.

Fragen

有問題嗎

คำถาม

?

質問

Вопросы

Questions

Câu hỏi